

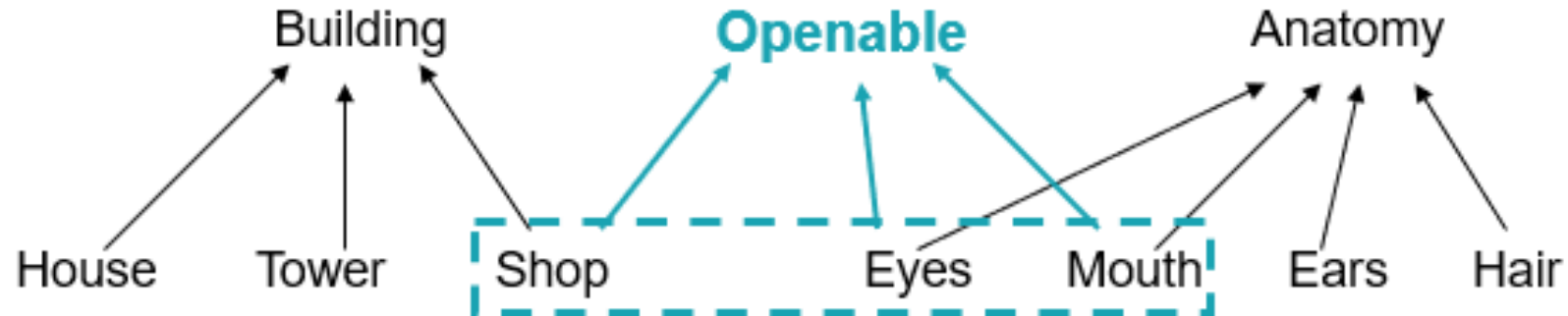
Interfaces

Introduction

- Java does not support multiple inheritance
 - Instead, it has the concept of an Interface
- Interfaces provide the power and leverage of multiple inheritance, but without the difficulties

An Example

- Consider these hierarchies



- In both hierarchies, there are classes which can be opened and closed

What is an Interface?

- An interface is like a totally abstract class
- It contains only abstract methods
 - And sometimes constant variables
 - And sometimes default method implementations (see later)

```
public interface Openable {  
    public abstract void open();  
    public abstract void close();  
}
```

What is in an Interface?

- Openable contains two abstract methods
 - open and close
- Methods are implicitly public and abstract, so you do not need to specify that part of the method signature

```
public interface Openable {  
    public abstract void open();  
    void close();  
}
```

Implementing Interfaces

- Classes implement interfaces using the *implements* keyword
- Classes can implement more than one interface separated by commas

```
public class Shop extends Building implements Openable, Taxable
```

```
public class Eye extends Anatomy implements Openable
```

```
public class Mouth extends Anatomy implements Openable
```

Implementing Interfaces Example

```
// implements Openable interface therefore
// must have an open and close method

public class Shop extends Building implements Openable {
    ....
    public void open() {
        // open the shop
    }

    public void close() {
        // close the shop
    }
    ....
}
```

Interfaces are Types

- You can exploit polymorphism with interfaces

```
Shop myShop = new Shop();  
Object yourShop = new Shop();  
Openable cornerShop = new Shop();
```

- You could have an array of Openable references

```
Openable[] myArray = { new Shop(),  
                        new Eye(),  
                        new Mouth() };
```

Which methods can be called?

- On references of type **Openable** we can call

- void open()
- void close()

✓

```
for (int i=0; i<myArray.length; i++)  
    myArray[i].open();
```

✓

```
cornerShop.close();
```

✗

```
cornerShop.raiseShutter();
```

- On references of interface types we can only call methods defined within the interface

Casting References

- If you have a reference of a type and want to use additional methods you can cast the reference to the appropriate type

```
Openable cornerShop = new Shop();  
Shop myShop = (Shop) cornerShop;  
myShop.raiseShutter();
```

- When casting, you can check the reference is the right type by using **instanceof**

```
if (cornerShop instanceof Shop)  
    Shop myShop = (Shop) cornerShop;
```

Default Methods

- Since Java 8, you can now provide a **default** implementation for any method in an interface
- Consider that Openable had a default way of doing close()
 - The method is marked as **default**

```
public interface Openable {  
    void open();  
    default void close() {  
        System.out.println("here is the default close");  
    }  
}
```

Default Method Implementation

- Classes implementing an interface with default methods no longer have to override the method
 - They can use the default

```
public interface Openable {  
    void open();  
    default void close() {  
        ..  
    }  
}
```

```
public class Shop extends Building  
    implements Openable {  
  
    public void open() {  
        // open the shop  
    }  
    // no need for close  
}
```

Summary

- Interfaces are totally abstract classes
- They typically contain abstract methods and can contain final static variables
- Implementation can be anywhere in the hierarchy
- Interface names are also a type